DFSha — Fase II

Revisión técnica del prototipo grupal · opción 1: cliente/servidor S2S

Andrés Felipe Martínez Velasquez · Juan José García Duque · Samuel Parra Cadavid

Universidad Pontificia Bolivariana · Profesor Álvaro Ospina Sanjuan

Estado: correcciones y validación local. Pendiente ratificación del equipo, aprobación de simplificaciones cuando aplique y repetición de evidencias en AWS Academy.

1. Alcance y estado verificable

Esta revisión corrige el prototipo tras contrastarlo con la consigna y los tres diseños individuales. Las pruebas de esta versión se hicieron en Docker local; la validación en AWS Academy está pendiente. Las capturas y mediciones de la versión anterior se conservan como históricas, con sus anomalías señaladas.

2. Comparación y elección arquitectónica

Criterio	Andrés	García	Samuel	Revisión
Control	2 APIs + etcd/Raft	PostgreSQL activo/pasivo	3 coordinadores + etcd	2 APIs + 3 etcd
Bloque	8 MiB	4 MiB	4 MiB	8 MiB; comparación pendiente
Réplicas	R3/W2	R2/W2	R3/W2	R3/W2, recibos verificados
Concurrencia	Por bloque	Bloque + versión esperada	Por archivo + CAS	rw por bloque; w/a exclusivos
Lectores	Snapshot; gracia	Manifiesto estable	Lectores renovables	Lease + retención por referencia
Seguridad	Argon2id/RS256/mTLS	Argon2id/RS256/mTLS	Argon2id/TLS/mTLS	scrypt/EdDSA/identidades separadas

La topología de Andrés es una base defendible por disponibilidad del catálogo y paralelismo. Se incorporan la validación de base de García y los leases lectores de Samuel. S2S y snapshots son decisiones coincidentes: no se atribuyen exclusivamente a una persona. El detalle con secciones originales y acuerdo pendiente está en COMPARACION_PROPUESTAS.md.

3. Decisiones y límites del contrato

Se mantiene REST/HTTPS para control, gRPC/mTLS para datos, tres etcd [3], dos APIs equivalentes y cuatro nodos, con separación de planos inspirada en GFS/HDFS [1], [2]. Los bloques miden hasta 8 MiB; el objetivo es R=3 y la preparación exige dos recibos de nodos distintos. La réplica S2S se hace desde un bloque validado, secuencialmente: no se presenta como pipeline solapado. ADR_REVISION.md documenta los cambios y sus costos.

La publicación usa un epoch global y CAS de sesión/referencias. Esto prioriza corrección, pero introduce contención de metadatos y no promete escalabilidad ilimitada. MAX_BLOCKS=8192; lock y rename atómicos hasta 48 bloques/entradas. El catálogo v2 no migra automáticamente el formato previo.

4. Componentes y despliegue

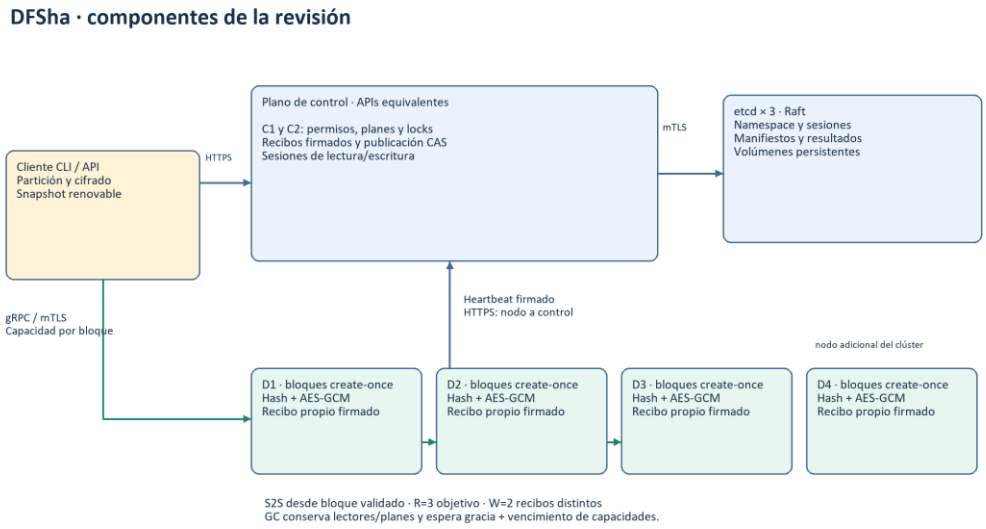


Figura 1. Componentes corregidos: nodos sin acceso al catálogo, recibos firmados y lectores con lease.

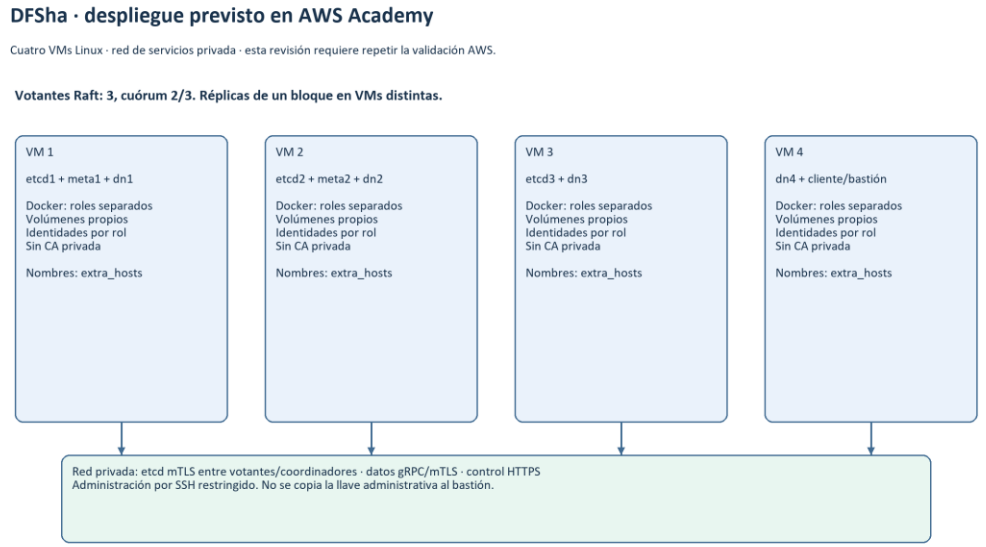


Figura 2. Despliegue previsto; la ejecución local no sustituye la demostración distribuida en AWS.

5. Protocolos, datos y algoritmos

Estado/operación	Contrato revisado
open	Valida propietario; fija snapshot; crea sesión con lease de 60 s renovable.
Reserva	Asigna ID nuevo y destinos; registra plan por bloque.
authorize	Fija SHA-256 y tamaño cifrado; emite capacidad ligada a operación y nodos.
WriteBlock	Consume stream incremental, valida, fsync, create-once y firma recibo por nodo.
prepared	Verifica firmas, nodos distintos y contenido del plan; persiste ficha preparada.
close	Valida planes/locks, tamaños y CAS; publica versión y resultado idempotente en txn acotada.
read	Lee snapshot; refresca ubicaciones/capacidades sin cambiar versión; valida hashes y GCM.
GC	Retiene referencias actuales, lectores y planes; marca, espera y valida antes del borrado.

RF1 se cubre con login y el árbol por usuario; RF2 con put/get de archivo completo; RF3 con open/close/read/write/lock y la semántica declarada: create-once por bloque, rw con lock, w/a exclusivos y append sin pasar EOF.

API: /v1/auth/login; /v1/fs/ls,mkdir,rmdir,rm,rename; /v1/files/open; /v1/files/{fd}/keepalive,manifest,lock,unlock,blocks,close,abort,status. Las rutas /blocks/{bid}/authorize y /prepared separan autorización y confirmación. El contrato gRPC [4] está en proto/dfsha.proto.

Ejemplo: POST /v1/files/{fd}/blocks con {"index":0,"request_id":"op-b0"}; authorize fija hash y tamaño; WriteBlock devuelve recibos firmados; prepared recibe {"receipts":["firma-dn1","firma-dn2"]}. Un recibo duplicado/falso recibe 409 BAD_RECEIPT. close con el mismo fd/cuerpo devuelve la versión publicada, sin publicar otra.

6. Consistencia, concurrencia y fallos

rw comprueba el ID base de cada bloque al adquirir el lock. Si otro escritor cambió ese bloque desde open, responde 409 y exige reabrir/releer; bloques distintos pueden avanzar en paralelo. w y a son exclusivos respecto a otros escritores. rm/rename rechazan escritores activos con 423. La expiración de la sesión elimina sus locks; el commit compara la revisión de sesión y no puede publicar con una sesión vencida.

Un lector abierto conserva snapshot con lease renovable [11]; la visibilidad entre aperturas sigue el modelo close-to-open [5]. Al cerrar o vencer, sus referencias dejan de proteger bloques obsoletos. La gracia no sustituye el seguimiento de lectores. El GC no elimina en el ciclo de marcado y espera el horizonte de capacidades para no permitir un replay tras borrar el ID.

Heartbeats firmados cada 3 s: sospechoso a 10 s y caído a 30 s. El monitor renueva liderazgo en paralelo, verifica que siga vigente y replica usando el hash autorizado del catálogo. Una fuente corrupta no se registra como reparada. Raft [9] mantiene el catálogo; con pérdida de cuórum, las operaciones fallan cerrado, priorizando consistencia [10]. No se promete disponibilidad ante cualquier combinación de fallos.

7. Seguridad e integridad

Cada servicio tiene certificado propio. La CA de etcd solo habilita miembros y coordinadores; nodos y cliente no reciben sus credenciales ni KEK. gRPC exige mTLS y capacidad. JWT y capacidades tienen firmantes separados, y cada nodo firma sus recibos. La KEK se carga solo en metadatos. El hash de contraseña usa scrypt con sal (N=32768,r=8,p=3). Estos cambios deben revisarse con el equipo y el profesor cuando corresponda.

AES-256-GCM [8] usa AAD=block_id; JWT [6] transporta identidad y capacidades, sobre canales TLS [7]. Cada bloque se valida por tamaño/hash; todas las versiones tienen hash canónico de manifiesto. put/w añade el SHA-256 plano global; rw/a no lo recalculan y lo declaran null. El hash de manifiesto no se presenta como hash de todo el archivo plano. Descargas usan temporal y solo reemplazan destino tras validar.

8. Pruebas ejecutadas y evidencia versionada

La corrida final (revision-20261103-062535) ejecutó 16 casos con 0 fallos, 0 errores y 0 omitidos; el código probado tiene hash 9632b34fadbb8ad6. La suite cubre recibos falsos y duplicados, contenido inmutable, snapshot sin caché, base vieja, bloques distintos, truncado vacío, rm/rename, idempotencia, rangos, aislamiento por usuario y transferencia de 129 bloques (1032 MiB) con hash coincidente. El registro completo está en el mismo directorio.

Los seis chequeos de revision-checks-20261103T064125Z terminaron en 0: análisis sintáctico de 21 archivos, seis pruebas de contrato y estadística, aislamiento de secretos por rol en los cuatro paquetes de despliegue, renderizado de las configuraciones Compose de AWS, identificador de imagen y rechazo de gRPC sin certificado de cliente. La prueba de caídas (revision-failures-8360fe3f7eb4) detiene dn1, meta1 y etcd1, comprueba lectura y escritura con el nodo caído, la reparación, el failover de API y el rechazo con 503 al perder el cuórum, y restaura los servicios al final. Son contenedores locales: las caídas de VM, la red privada y las mediciones finales deben repetirse en AWS Academy.

9. Métricas reproducibles

El benchmark de revision-bench-20261103T062822Z-73ae1d guarda las muestras crudas, el resumen, los parámetros y el hash del código. Cada fila usa cinco repeticiones sobre un archivo de 10 MiB; en E3, dos clientes suben 20 MiB en total y la tasa es agregada.

Exp.	Condición	Operación	Media (s)	p95 (s)	MiB/s
E1	R=3	put	0.99	1.23	10.3
E1	R=3	get	0.53	0.64	20.1
E2	R=2, W=2	put	0.92	1.02	11.0
E2	R=3, W=2	put	0.98	1.05	10.2
E3	1 cliente	put	0.95	0.98	10.5
E3	1 cliente	get	0.43	0.47	23.3
E3	2 clientes	put	1.35	1.46	14.8
E3	2 clientes	get	0.60	0.72	34.3

Con un cliente, put ronda 0.99 s y get 0.53 s. R=2 mejora el put cerca de 7 % frente a R=3 (11.0 vs 10.2 MiB/s). Dos clientes elevan la tasa agregada, pero la latencia por operación sube; no se presenta como escalabilidad lineal. Con cinco muestras el p95 es el máximo observado: el resultado sirve como prueba de humo y no para caracterizar colas. El bench.csv anterior se descarta por su inconsistencia entre media y p95.

Pendiente en AWS: 10/100/500 MiB y borde 1 GiB, más clientes, comparación 3/4 nodos, 4/8 MiB en clústeres separados, disponibilidad, tiempo de reparación y CPU/memoria/disco/NIC. No se atribuye un cuello de botella sin esas mediciones.

10. Escenarios obligatorios de razonamiento

Escenario	Respuesta y comprobación
Mismo bloque simultáneo	Un lock excluye al segundo escritor (423). Si abrió sobre base anterior, se rechaza con 409 al adquirir. Se preserva contenido; se probó la base vieja y bloques distintos.
Réplica desconectada	La publicación exige dos recibos distintos, firmados y persistentes. Una sola copia no basta. El monitor repara con hash autorizado; los lectores usan réplicas de su snapshot.
Caída a mitad de bloque	El temporal no obtiene recibo ni se publica. La regresión envía un stream incompleto, comprueba DATA_LOSS y rechazo del commit. Reintento usa un plan nuevo.
Ubicación desconocida	El cliente usa nombres lógicos de coordinadores; open/manifest devuelve destinos y capacidades. El refresco conserva el mismo snapshot. extra_hosts se configura dentro de cada contenedor.

11. Entrega, contribuciones y defensa

Cada integrante registra sus aportes mediante commits en el repositorio del equipo. El guion del video de 10–15 minutos y la preparación de la defensa están en informe/GUION_VIDEO.md e informe/GUION_DEFENSA.md. Faltan la evidencia en AWS de esta versión y las aprobaciones aplicables.

Referencias y fuentes de comparación

Se conservan las referencias del informe previo sin atribuirles resultados experimentales nuevos. Los documentos originales individuales son las fuentes de la comparación; las diapositivas deben estar disponibles para el equipo.

[1] S. Ghemawat, H. Gobioff y S.-T. Leung, "The Google file system," en Proc. 19th ACM Symp. Operating Systems Principles, 2003, pp. 29-43.

[2] K. Shvachko, H. Kuang, S. Radia y R. Chansler, "The Hadoop distributed file system," en Proc. IEEE MSST, 2010, pp. 1-10.

[3] etcd Authors, "etcd documentation," 2026. Disponible: https://etcd.io/docs/

[4] A. Ospina Sanjuan, "gRPC: de la invocación local a los microservicios modernos," diapositivas de clase, Universidad Pontificia Bolivariana, 2026.

[5] J. H. Howard et al., "Scale and performance in a distributed file system," ACM Trans. Computer Systems, vol. 6, no. 1, pp. 51-81, 1988.

[6] M. Jones, J. Bradley y N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[7] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, IETF, 2018.

[8] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, 2007.

[9] D. Ongaro y J. Ousterhout, "In search of an understandable consensus algorithm," en Proc. USENIX ATC, 2014, pp. 305-319.

[10] E. Brewer, "CAP twelve years later: How the 'rules' have changed," Computer, vol. 45, no. 2, pp. 23-29, 2012.

[11] M. van Steen y A. S. Tanenbaum, Distributed Systems, 4.ª ed. distributed-systems.net, 2023.